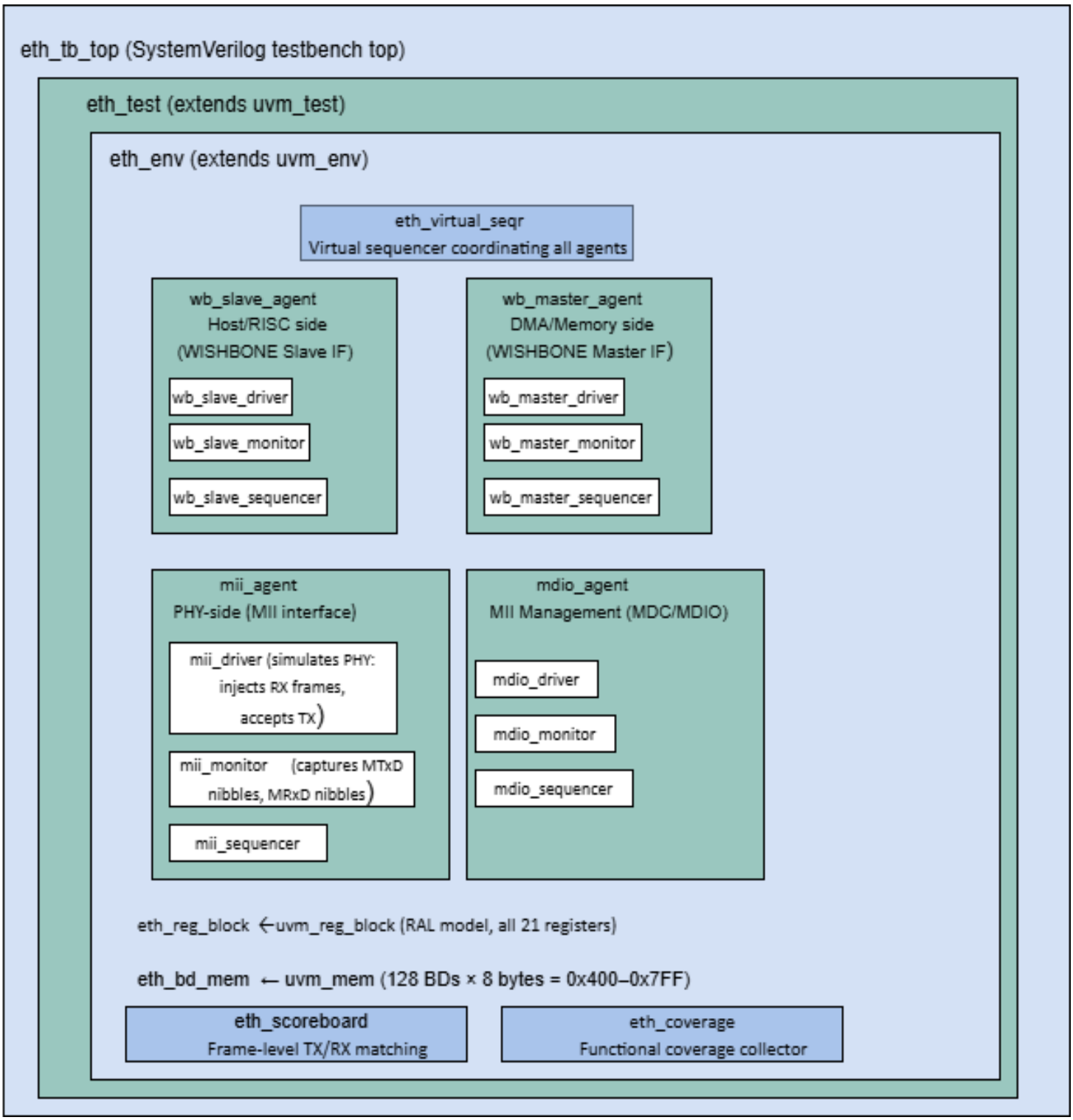
Ethernet IP Core verification environment

1.Top-Level UVM Architecture



2. Interface Definitions

We need **four** SystemVerilog interfaces:

Interface	Signals
`wb_if` (slave)	CLK_I, RST_I, ADDR_I, DATA_I/O, SEL_I, WE_I, STB_I, CYC_I, ACK_O, ERR_O, INTA_O
`wb_if` (master)	M_ADDR_O, M_DATA_I/O, M_SEL_O, M_WE_O, M_STB_O, M_CYC_O, M_ACK_I, M_ERR_I
`mii_if`	MTxClk, MTxD[3:0], MTxEn, MTxErr, MRxClk, MRxD[3:0], MRxDV, MRxErr, MColl, MCrS
`mdio_if`	MDC, MDIO (bidirectional), Mdi, Mdo, MdoEn

The Slave Interface — Configuration Port

This is how a processor (or testbench) **configures the MAC**. The DUT is the **slave** here

Used for:

- Writing configuration registers (MODER, PACKETLEN, COLLCONF, etc.)
- Reading status registers (INT_SOURCE, MIISTATUS, etc.)
- Reading/writing Buffer Descriptors in internal RAM (0x400–0x7FF)

The DUT **receives** commands on this interface. It never initiates anything here.

Signals on this interface (from spec Table 1):

INPUT into DUT (driven by testbench): OUTPUT from DUT:

CLK_I — clock

RST_I — reset

ADDR_I — register/BD address

DATA_I — data to write

SEL_I — byte enables (must be 1111)

WE_I — 1=write, 0=read

STB_I — "this transfer is valid"

CYC_I — "a bus cycle is in progress"

ACK_O — "I'm done, transaction OK"

ERR_O — "bad access (e.g. SEL≠1111)"

INTA_O — interrupt to processor

DATA_O — data read back

The Master Interface — DMA Port

This is how the MAC **autonomously moves frame data** to/from system memory. The DUT is the **master** here — it initiates transactions without being asked.

Used for:

- TX path: DUT reads frame bytes from memory to fill its TX FIFO
- RX path: DUT writes received frame bytes into memory from its RX FIFO

The DUT **initiates** all transactions on this interface. testbench must respond as a slave.

Signals on this interface (from spec Table 1):

OUTPUT from DUT (DUT initiates):	INPUT into DUT (testbench responds):
M_ADDR_O — memory address to access	M_ACK_I — "memory access done"
M_DATA_O — data to write to memory	M_ERR_I — "memory access error"
M_SEL_O — byte enables	M_DATA_I — data read from memory
M_WE_O — 1=write(RX), 0=read(TX)	
M_STB_O — "this transfer is valid"	
M_CYC_O — "a bus cycle is in progress"	

---

wb_slave_agent

└── Connects to: WISHBONE SLAVE interface of DUT

└── Driven by: testbench (acts as the RISC processor)

└── Purpose: Write registers, set up BDs, read status, clear interrupts

└── Signals: CLK_I, RST_I, ADDR_I, DATA_I, DATA_O, SEL_I,

WE_I, STB_I, CYC_I, ACK_O, ERR_O, INTA_O

wb_master_agent

└── Connects to: WISHBONE MASTER interface of DUT

└── Driven by: testbench (acts as the memory/SRAM)

└── Purpose: Respond to DMA reads (provide TX frame data) and
DMA writes (accept RX frame data into its SRAM model)

└── Signals: M_ADDR_O, M_DATA_O, M_DATA_I, M_SEL_O, M_WE_O,
M_STB_O, M_CYC_O, M_ACK_I, M_ERR_I

mii_agent

└── Connects to: MII interface of DUT

└── Driven by: Your testbench (acts as the Ethernet PHY chip)

└── Purpose: Inject RX frames (drive MRxD nibbles + MRxDV),
capture TX frames (sample MTxD + MTxEn),
inject collisions (MColl) and carrier sense (MCrS)

└── Signals: MTxClk, MTxD[3:0], MTxEn, MTxErr,
MRxClk, MRxD[3:0], MRxDV, MRxErr, MColl, MCrS

mdio_agent

└── Connects to: MDC/MDIO interface of DUT

└── Driven by: testbench (acts as the PHY's management registers)

└── Purpose: Respond to PHY register reads/writes initiated by DUT,
verify correct MIIM frame format on MDIO

└── Signals: MDC (input, DUT drives), MDIO (bidirectional)

3. RAL Model — `eth_reg_block`

```
class eth_reg_block extends uvm_reg_block;

  // All 21 registers

  rand eth_moder_reg        MODER;        // 0x00

  rand eth_int_source_reg   INT_SOURCE;   // 0x04

  rand eth_int_mask_reg     INT_MASK;     // 0x08

  rand eth_ipgt_reg         IPGT;         // 0x0C

  rand eth_ipgr1_reg        IPGR1;        // 0x10

  rand eth_ipgr2_reg        IPGR2;        // 0x14

  rand eth_packetlen_reg    PACKETLEN;    // 0x18

  rand eth_collconf_reg     COLLCONF;     // 0x1C

  rand eth_tx_bd_num_reg    TX_BD_NUM;    // 0x20

  rand eth_ctrlmoder_reg    CTRLMODER;    // 0x24

  rand eth_miimoder_reg     MIIMODER;     // 0x28

  rand eth_miicommand_reg   MIICOMMAND;   // 0x2C

  rand eth_miiaddress_reg   MIIADDRESS;   // 0x30

  rand eth_miitx_data_reg   MIITX_DATA;  // 0x34

  rand eth_miirx_data_reg   MIIRX_DATA;  // 0x38  (R only)

  rand eth_miistatus_reg    MIISTATUS;   // 0x3C  (R only)

  rand eth_mac_addr0_reg    MAC_ADDR0;   // 0x40

  rand eth_mac_addr1_reg    MAC_ADDR1;   // 0x44

  rand eth_hash0_reg        HASH0;        // 0x48

  rand eth_hash1_reg        HASH1;        // 0x4C
```

```
  rand eth_txctrl_reg       TXCTRL;       // 0x50


  // BD memory: 0x400–0x7FF
  uvm_mem eth_bd_mem;  // 256 × 32-bit words
  uvm_reg_map reg_map;
endclass
```

Each register class models its named fields. For example, `MODER` has fields: `RECSMALL`, `PAD`, `HUGEN`, `CRCEN`, `DLYCRCEN`, `FULLD`, `EXDFREN`, `NOBCKOF`, `LOOPBCK`, `IFG`, `PRO`, `IAM`, `BRO`, `NOPRE`, `TXEN`, `RXEN` — each with correct access type and reset value from the spec.

4. Complete Test Plan

Group 1: Register & BD Access Tests

Test	Description	RAL Method
tc_reg_reset_values	Verify all 21 registers match spec reset values after RST_I	uvm_reg_hw_reset_seq
tc_reg_walking_one	Walking-1 across all RW bits, single WISHBONE cycles	uvm_reg_bit_bash_seq
tc_reg_walking_zero	Walking-0 across all RW bits	uvm_reg_bit_bash_seq
tc_reg_max_values	Write max value, read back, hard reset, verify	Custom seq
tc_reg_ro_fields	Verify R-only fields (MIIRX_DATA, MIISTATUS bits) reject writes	Custom seq
tc_bd_walking_one	Walking-1 across BD RAM (0x400–0x7FF)	uvm_mem_walk_seq
tc_bd_reset_preservation	BD RAM holds values after soft reset (RST bit), clears after hard reset	Custom seq
tc_bd_boundary	Access first BD (0x400), last BD (0x7FC), and out-of-range	uvm_mem_access_seq
tc_tx_bd_num_split	Write TX_BD_NUM=0x32 → verify 50 TX BDs and 78 RX BDs are accessible	Custom seq
tc_bd_access_during_reset	Attempt BD access while RST bit set → expect no access	Custom seq

---

Group 2: MIIM (MII Management) Tests

Test	Description
tc_miim_clkdiv_all	Test all 255 clock divider values in MIIMODER.CLKDIV; verify MDC frequency = CLK/(2×CLKDIV)
tc_miim_write_phy_reg	Write to PHY control register (addr 0x0) via MIICOMMAND.WCTRLDATA; verify MDIO frame format
tc_miim_read_phy_reg	Read from PHY status register (addr 0x1); verify PRSD in MIIRX_DATA
tc_miim_read_nonwritable	Attempt write to read-only PHY register; verify PHY ignores it
tc_miim_reset_phy	Assert PHY reset through MIIM write; verify PHY responds
tc_miim_walk_phy_addr	Walking-1 across FIAD[4:0] with and without preamble
tc_miim_walk_reg_addr	Walking-1 across RGAD[4:0] with and without preamble
tc_miim_walk_data	Walking-1 across 16-bit write data with and without preamble
tc_miim_wrong_phy_addr	Read from non-existent PHY address → MDIO high-Z, verify MIIRX_DATA undefined
tc_miim_wrong_write_correct_read	Write to wrong PHY addr, read from correct addr; verify no corruption
tc_miim_busy_nvalid_write	Verify BUSY asserts during write, deasserts at end; NVALID behavior
tc_miim_busy_nvalid_read	Verify BUSY/NVALID durations during read operation
tc_miim_busy_nvalid_scan	Verify BUSY stays asserted during continuous scan; NVALID clears after first scan
tc_miim_scan_linkfail	Scan PHY status; detect LINKFAIL asserted/deasserted; verify MIISTATUS.LINKFAIL
tc_miim_scan_linkfail_sliding	Sliding LINKFAIL bit during scan — verify edge detection
tc_miim_stop_scan_after_read	Assert SCANSTAT then immediately clear; sliding stop with/without preamble
tc_miim_stop_scan_after_write	Stop scan immediately after write request; sliding
tc_miim_stop_scan_after_scan	Stop scan after 1st and 2nd scan iterations; sliding
tc_miim_no_preamble	Set MIIMODER.MIINOPRE=1; verify 32-bit preamble absent on MDIO

Group 3: Frame Transmission Tests

Test	Description
tc_tx_basic_frame	Transmit single minimum-length frame (64B); verify preamble + SFD + data + CRC on MII
tc_tx_max_frame	Transmit maximum-length frame (1518B); verify no truncation
tc_tx_short_frame_pad_on	Frame < MINFL with PAD=1; verify padded to 64B with zeros + CRC
tc_tx_short_frame_pad_off	Frame < MINFL with PAD=0; verify transmitted as-is (no padding)
tc_tx_crc_enable	CRCEN=1; verify 4-byte CRC appended to frame
tc_tx_crc_disable	CRCEN=0; verify no CRC appended
tc_tx_delayed_crc	DLYCRCEN=1; CRC calculation starts 4 bytes after SFD
tc_tx_huge_frame	HUGEN=1; transmit frame > 1518B; verify transmission completes
tc_tx_huge_disabled	HUGEN=0; transmit frame > MAXFL; verify abort at max
tc_tx_no_preamble	NOPRE=1 in MODER; verify 7-byte preamble suppressed
tc_tx_ipg_back_to_back	Transmit back-to-back frames; verify IPG ≥ 0.96μs (IPGT register value)
tc_tx_ipg_non_back_to_back	Verify IPGR1 and IPGR2 windows respected
tc_tx_multi_bd_chain	Chain 8 TX BDs (WR=0 until last); verify continuous transmission
tc_tx_wrap_bit	Last BD has WR=1; verify circular wrap to first BD
tc_tx_underrun	Throttle WISHBONE master to starve TX FIFO; verify TxAbort + UR bit in TX BD
tc_tx_done_interrupt	IRQ=1 in TX BD; verify TXB interrupt in INT_SOURCE after transmission
tc_tx_error_interrupt	Force underrun with IRQ=1; verify TXE interrupt
tc_tx_retry_count	In half-duplex: inject collision; verify RTRY field incremented in TX BD
tc_tx_max_retry	Inject MAX_RET+1 collisions; verify RL bit set, transmission aborted
tc_tx_late_collision	Inject collision after COLLVALID window; verify LC bit + abort
tc_tx_excessive_deferral	EXDFREN=0; keep carrier busy > defer limit; verify abort
tc_tx_excessive_deferral_wait	EXDFREN=1; verify TX waits indefinitely for carrier
tc_tx_loopback	LOOPBCK=1; verify transmitted data appears on RX path
tc_tx_carrier_sense_lost	Assert MCrS then deassert mid-frame; verify CS bit in TX BD
tc_tx_only_4byte_min	TX BD with LEN ≤ 4; verify frame not transmitted

	(spec note)
tc_tx_unaligned_ptr	TX buffer pointer at non-word-aligned address; verify correct byte extraction

---

Group 4: Frame Reception Tests

Test	Description
tc_rx_basic_frame	PHY sends standard frame; verify assembly and storage in memory via DMA
tc_rx_crc_check_pass	Valid CRC frame; verify no CRC error in RX BD
tc_rx_crc_check_fail	Corrupt last byte of CRC; verify CRC bit set in RX BD
tc_rx_short_frame_reject	Frame < MINFL; RECSMALL=0; verify frame dropped
tc_rx_short_frame_accept	Frame < MINFL; RECSMALL=1; verify SF bit set in RX BD
tc_rx_too_long_frame	Frame > MAXFL; HUGEN=0; verify reception stops at MAXFL; TL bit set
tc_rx_huge_frame_enable	Frame > MAXFL; HUGEN=1; verify full frame received
tc_rx_preamble_removal	Verify 7-byte preamble + SFD stripped; only payload in memory
tc_rx_short_preamble	PHY sends frame with <7-byte preamble (only SFD); verify accepted
tc_rx_delayed_crc	DLYCRCEN=1; verify CRC checking starts 4 bytes after SFD
tc_rx_dribble_nibble	PHY sends odd-nibble-count frame; verify DN and CRC bits in RX BD
tc_rx_invalid_symbol	PHY sets MRxD=0xE (100Mbps error code); verify IS bit in RX BD
tc_rx_phy_error	MRxErr asserted mid-frame; verify frame aborted, no error in BD
tc_rx_overrun	Throttle WISHBONE master; RX FIFO overrun; verify OR bit in RX BD
tc_rx_ifg_minimum	Two back-to-back frames with IFG < 960ns; verify second frame dropped
tc_rx_ifg_disable	r_IFG=1 in MODER; verify frames accepted regardless of IFG
tc_rx_multi_bd	Receive frames filling multiple RX BDs; verify wrap using WR bit
tc_rx_no_empty_bd	All RX BDs marked non-empty; verify frame discarded, BUSY interrupt
tc_rx_interrupt_rxb	IRQ=1 in RX BD; verify RXB interrupt after frame received
tc_rx_interrupt_rxe	Receive with error, IRQ=1; verify RXE interrupt
tc_rx_late_collision	Late collision during reception; verify LC bit in RX BD
tc_rx_unaligned_ptr	RX buffer pointer at non-word-aligned address; verify

	correct byte select
tc_rx_only_4byte_min	Frame ≤ 4 bytes; verify received with CRC error (per spec note)

Group 5: Address Recognition Tests

Test	Description
tc_addr_unicast_match	Frame dest = MAC_ADDR0/1; verify accepted
tc_addr_unicast_mismatch	Frame dest ≠ MAC address; PRO=0; verify dropped (RxAbort)
tc_addr_promiscuous	PRO=1; send frame with foreign dest; verify accepted + M bit set in RX BD
tc_addr_broadcast_accept	BRO=0; send broadcast (FF:FF:FF:FF:FF:FF); verify accepted
tc_addr_broadcast_reject	BRO=1; send broadcast; PRO=0; verify rejected
tc_addr_multicast_hash	IAM=1; load HASH0/HASH1; send multicast matching hash; verify accepted
tc_addr_multicast_hash_miss	IAM=1; multicast not in hash table; verify dropped
tc_addr_miss_bit	PRO=1; frame doesn't match MAC; verify M=1 in RX BD

Group 6: Flow Control Tests (Full Duplex)

Test	Description
tc_fc_rx_pause_frame	PHY sends valid PAUSE frame (type=8808, op=0001); RXFLOW=1; verify TX paused for timer value
tc_fc_rx_pause_timer_expire	PAUSE timer counts down to 0; verify TX resumes
tc_fc_rx_pause_passall_0	PASSALL=0, RXFLOW=1; PAUSE frame not stored to memory; RXC interrupt set
tc_fc_rx_pause_passall_1	PASSALL=1, RXFLOW=1; PAUSE frame stored to memory + CF bit set + RXC interrupt
tc_fc_rx_pause_passall_1_rxflow_0	PASSALL=1, RXFLOW=0; stored as normal frame, RXB set, no pause
tc_fc_rx_pause_both_0	PASSALL=0, RXFLOW=0; PAUSE frame silently dropped
tc_fc_tx_pause_request	Write TXPAUSERQ=1 to TXCTRL with TXPAUSETV; verify PAUSE frame transmitted with correct timer value
tc_fc_tx_pause_frame_format	Verify PAUSE frame structure: dest=01-80-c2-00-00-01, type=8808, op=0001, 42B reserved,

	CRC
tc_fc_tx_pause_blocked	TXFLOW=0; write TXPAUSERQ=1; verify no PAUSE frame sent
tc_fc_rx_invalid_pause	Receive frame with wrong type/opcode; verify not treated as PAUSE
tc_fc_slot_timer	Verify slot timer generates correct pulse intervals (64-byte slot time at 10/100 Mbps)
tc_fc_txc_interrupt	TXFLOW=1; send PAUSE frame; verify TXC interrupt in INT_SOURCE
tc_fc_rxc_interrupt	RXFLOW=1; receive PAUSE frame; verify RXC interrupt in INT_SOURCE

Group 7: Half-Duplex / CSMA-CD Tests

Test	Description
tc_hd_collision_backoff	FULLD=0; inject collision; verify jam pattern (0x99999999) + binary exponential backoff
tc_hd_collision_retry	Inject up to 15 collisions; verify each retry uses wider backoff window; TxRetry signal
tc_hd_collision_window	Verify ColWindow = COLLVALID bytes from preamble
tc_hd_nobckof	NOBCKOF=1; verify immediate retransmission after collision (no backoff)
tc_hd_carrier_sense_defer	MCrS asserted; verify TX defers; DF bit set after eventual success
tc_hd_carrier_sense_ifg	Carrier appears within IPGR1 window; verify counter reset

---

Group 8: Interrupt System Tests

Test	Description
tc_int_mask_all	INT_MASK=0x00; trigger all events; verify INTA_O never asserts
tc_int_unmask_txb	INT_MASK.TXB_M=1; transmit frame; verify INTA_O asserts
tc_int_unmask_rxb	INT_MASK.RXF_M=1; receive frame; verify INTA_O asserts
tc_int_clear_by_write1	Write 1 to INT_SOURCE bits to clear; verify bits clear and INTA_O deasserts
tc_int_busy	Fill all RX BDs; receive new frame; verify BUSY bit in INT_SOURCE
tc_int_multiple_sources	Trigger TX + RX simultaneously; verify all bits set, single INTA_O

Group 9: Reset & Clock Domain Tests

Test	Description
tc_rst_hard_reset	Assert RST_I; verify all registers return to reset values; TX/RX halt
tc_rst_mid_tx	Assert RST_I during frame transmission; verify graceful abort
tc_rst_mid_rx	Assert RST_I during frame reception; verify BD not corrupted
tc_rst_bd_preservation	BD RAM holds values after RST_I (per design doc section 3.4.1)
tc_clk_10mbps	Set PHY clocks to 2.5MHz (10Mbps); full TX/RX test
tc_clk_100mbps	Set PHY clocks to 25MHz (100Mbps); full TX/RX test
tc_clk_no_link	MRxClk random 2-40MHz (no link); verify MAC does not lock up

---

Group 10: Stress & Corner Case Tests

Test	Description
tc_stress_all_bd_tx	Fill all 128 TDs as TX BDs (TX_BD_NUM=0x80); transmit burst
tc_stress_all_bd_rx	TX_BD_NUM=0x00; all BDs as RX; receive burst
tc_stress_simultaneous_txrx	Full duplex: simultaneous TX burst and RX burst
tc_stress_wb_wait_states	Add random wait states to WISHBONE slave; verify no deadlock
tc_stress_bd_boundary_addr	Access TX BDs at 0x400, RX BDs ending at 0x7FC; boundary check
tc_stress_random_frame_size	Randomize frame sizes from 5 to 1600 bytes; run 1000 frames
tc_stress_fifo_depth	Fill TX FIFO to capacity (16 words) then drain; verify no stall

1- tc_reg_reset_values

After asserting RST_I, every one of the 21 configuration registers must read back its documented reset value as per the specification. Uses the UVM RAL built-in reset sequence so no manual register reads are needed.

COMPONENTS USED

`wb_slave_agent` `eth_reg_block (RAL)` `uvm_reg_hw_reset_seq` `eth_coverage`

TEST FLOW

1-TestAssert `RST_I=1` for 5 clock cycles via `wb_slave_agent`

2-`Test`Deassert `RST_I=0`, wait 2 clock cycles
3-`uvm_reg_hw_reset_seq`Iterates every register in `eth_reg_block`; calls `reg.read()` via frontdoor WISHBONE
4-`wb_slave_driver`Drives `ADDR_I, WE_I=0, STB_I, CYC_I`; waits for `ACK_O`
5-`RAL predictor`Compares read value against `reg.get_reset()` for every field
6-`eth_coverage`Samples `reg_reset_cg` covergroup — all register names

PASS CRITERIA

MODER reset = 0x0000_A000 (NOPRE, NOBCKOF, FULLD bits check)
IPGT reset = 0x12, IPGR1 = 0x0C, IPGR2 = 0x12
PACKETLEN reset = 0x0040_0600
COLLCONF reset = 0x000F_003F
TX_BD_NUM reset = 0x40
MIIMODER reset = 0x64
All status/interrupt regs (INT_SOURCE, MIISTATUS) reset = 0x0
uvm_reg_hw_reset_seq reports zero mismatches

`uvm_reg_hw_reset_seq`

It is a **pre-built UVM sequence**

Its entire job is one thing:

Read every register in your RAL model via the actual bus (frontdoor), compare each value against the reset value your RAL model declares, and report any mismatch as a UVM error.

The Actual Source Code (Simplified)

Here is what the sequence actually does internally, stripped to its essence:

systemverilog

```
class uvm_reg_hw_reset_seq extends uvm_reg_sequence;

  uvm_reg_block model;   // you set this — points to your eth_reg_block

  task body();

    uvm_reg regs[$];          // queue to hold all register handles
    uvm_reg_data_t rdata;     // value read from hardware
    uvm_reg_data_t exp;       // expected reset value from RAL model
    uvm_status_e   status;

    // Step 1: collect every register from the block
    model.get_registers(regs);

    // Step 2: for each register
```

```
        foreach (regs[i]) begin

            // Step 3: skip registers with no reset value defined
            if (!regs[i].has_reset()) continue;

            // Step 4: read the register from actual hardware
            //         this goes through your adapter → WISHBONE bus
            regs[i].read(status, rdata, UVM_FRONTDOOR);

            // Step 5: get what the reset value SHOULD be
            exp = regs[i].get_reset();

            // Step 6: compare — only check bits that have reset values
            //         (reserved bits are masked out automatically)
            if ((rdata & regs[i].get_reset("HARD"))
                !== regs[i].get_reset("HARD")) begin

                `uvm_error("REG_HW_RESET",
                    $sformatf("Register '%s' expected 0x%0h after reset, got 0x%0h",
                            regs[i].get_full_name(), exp, rdata))
            end
        end

    endtask

endclass
```

For MODER (offset 0x00): CYC_I = 1 STB_I = 1 WE_I = 0 ← read SEL_I = 4'hF ADDR_I = 0x00

DUT responds: ACK_O=1, DATA_O = current register value

adapter captures DATA_O, returns to RAL

RAL compares against 0x0000_A000

2-tc_reg_walking_one

Shifts a single 1-bit through every RW-accessible bit position in every register. For each bit position: write 1 to that bit only, read back, verify; then restore to reset value. Uses uvm_reg_bit_bash_seq.

COMPONENTS USED

wb_slave_agent	eth_reg_block (RAL)	uvm_reg_bit_bash_seq

TEST FLOW

1uvm_reg_bit_bash_seqFor each RW register: writes all zeros (frontdoor), reads back, checks

2uvm_reg_bit_bash_seqFor each RW bit position N: writes `1 << N`, reads back

3wb_slave_driverEach write: drives ADDR_I=reg_offset, DATA_I=value, WE_I=1, STB_I, CYC_I; waits ACK_O

4RAL predictorVerifies read data matches written value; flags W1C or read-only bits that differ

5uvm_reg_bit_bash_seqWrites all ones, reads back; checks RO bits unchanged, RW bits set

PASS CRITERIA

Every RW bit in all 21 registers accepts 0→1 and 1→0 transitions

Read-only bits (MIIRX_DATA[15:0], MIISTATUS[2:0]) never change on write

No ERR_O asserted for valid register addresses with SEL_I=4'hF

ERR_O asserted when SEL_I ≠ 4'hF on any register access

What `uvm_reg_bit_bash_seq` Actually Does Internally

The UVM library sequence does more than just walking 1. It runs four sub-patterns on every register. Here is its actual internal logic:

systemverilog

```
// Inside uvm_reg_bit_bash_seq (UVM library)

task body();

   uvm_reg regs[$];
   model.get_registers(regs);    // collect all registers

   foreach (regs[i]) begin
      bash_kth_reg(regs[i]);    // test this register completely
   end

endtask


task bash_kth_reg(uvm_reg rg);


   int n_bits;
   uvm_reg_field fields[$];
   uvm_reg_data_t  dc_mask;     // don't-care mask (reserved + RO bits)
   uvm_reg_data_t  reset_val;

   n_bits    = rg.get_n_bytes() * 8;   // 32 for your registers
   reset_val = rg.get_reset();
   rg.get_fields(fields);

   // Build a mask of bits we should NOT touch or check
   // (reserved bits, read-only bits, write-only bits)
```

```
   dc_mask = 0;
   foreach (fields[i]) begin
      if (fields[i].get_access() != "RW")
         dc_mask |= fields[i].get_mask();
      //           ^ set those bit positions in the don't-care mask
   end

   // ── Pattern 1: Walking 1 ──────────────────────────────────────
   for (int k = 0; k < n_bits; k++) begin

      if (dc_mask[k]) continue;   // skip reserved/RO bits

      bash_bit(rg, k, 1, reset_val, dc_mask);
      //           ↑  ↑
      //         bit  value to write
   end

   // ── Pattern 2: Walking 0 ──────────────────────────────────────
   // First write all 1s to every RW bit
   rg.write(status, ~dc_mask);

   for (int k = 0; k < n_bits; k++) begin

      if (dc_mask[k]) continue;

      bash_bit(rg, k, 0, reset_val, dc_mask);
      //           ↑  ↑
      //         bit  value to write (0 this time)
   end

endtask


task bash_bit(uvm_reg    rg,
              int        k,          // which bit position
              bit        val,        // 1 for walking-1, 0 for walking-0
              uvm_reg_data_t reset_v,
              uvm_reg_data_t dc_mask);

   uvm_reg_data_t  wr_val;
   uvm_reg_data_t  rd_val;
   uvm_reg_data_t  exp_val;
   uvm_status_e    status;

   // ── Step A: Build the write value ──
   if (val == 1)
      wr_val = (1 << k);           // only bit k is 1, all others 0
   else
      wr_val = ~(1 << k) & ~dc_mask; // only bit k is 0, all RW bits 1

   // ── Step B: Write to hardware (frontdoor WISHBONE) ──
   rg.write(status, wr_val);

   // ── Step C: Read back from hardware (frontdoor WISHBONE) ──
   rg.read(status, rd_val);
```

```
    // ── Step D: Build expected value ──
    // For RW bits: expect exactly what you wrote
    // For RO/reserved bits: expect their current value (unchanged)
    exp_val = (wr_val & ~dc_mask) | (reset_v & dc_mask);

    // ── Step E: Compare ──
    if ((rd_val & ~dc_mask) !== (exp_val & ~dc_mask)) begin
        `uvm_error("REG_BIT_BASH",
          $sformatf("Reg '%s' bit %0d: wrote 0x%0h, expected 0x%0h, got 0x%0h",
                rg.get_full_name(), k, wr_val, exp_val, rd_val))
    end

    // ── Step F: Restore register to reset value ──
    rg.write(status, reset_v);

endtask
```

Walking Through a Concrete Example — MODER Register

MODER is 32 bits wide. Reset value = 0x0000_A000.

Let us map out which bits are RW vs reserved:

```
Bit 31–17:  Reserved   → dc_mask bits 31–17 = 1  (skip these)
Bit 16:     RECSMALL   → RW  → test this
Bit 15:     PAD        → RW  → test this
Bit 14:     HUGEN      → RW  → test this
Bit 13:     CRCEN      → RW  → test this  (reset=1)
Bit 12:     DLYCRCEN   → RW  → test this
Bit 11:     Reserved   → dc_mask bit 11 = 1  (skip)
Bit 10:     FULLD      → RW  → test this
Bit 9:      EXDFREN    → RW  → test this
Bit 8:      NOBCKOF    → RW  → test this  (reset=1)
Bit 7:      LOOPBCK    → RW  → test this
Bit 6:      IFG        → RW  → test this
Bit 5:      PRO        → RW  → test this
Bit 4:      IAM        → RW  → test this
Bit 3:      BRO        → RW  → test this
Bit 2:      NOPRE      → RW  → test this
Bit 1:      TXEN       → RW  → test this
Bit 0:      RXEN       → RW  → test this

dc_mask = 0xFFFF_0800   (bits 31–17 and bit 11)
```

Now the sequence walks through 32 bit positions for MODER. Let us trace every interesting one:

Bit 0 — RXEN

k = 0

Step A: wr_val = (1 << 0) = 0x0000_0001

Step B: WISHBONE write
ADDR_I = 0x00 (MODER offset)
DATA_I = 0x0000_0001
WE_I = 1
SEL_I = 0xF
STB_I = 1
CYC_I = 1
→ wait for ACK_O

Step C: WISHBONE read
ADDR_I = 0x00
WE_I = 0
→ DUT returns DATA_O

Step D: exp_val = (0x0000_0001 & ~0xFFFF0800)
| (0x0000A000 & 0xFFFF0800)
= 0x0000_0001 & 0x0000_F7FF
| 0x0000_A000 & 0xFFFF_0800
= 0x0000_0001
| 0x0000_A000 ← bits 13 and 8 are reset=1, in dc_mask? NO
wait — bits 13,8 are RW, not reserved
dc_mask only covers reserved bits
= 0x0000_0001
(reserved bits stay 0, RW bits we only wrote RXEN=1)

Step E: DUT read returns 0x0000_0001 → PASS ✓

Step F: restore: write 0x0000_A000 (MODER reset value)

Wait — step D needs clarification. The expected value for RW bits is exactly what you wrote. You wrote 0x0000_0001. Only RXEN=1. All other RW bits including CRCEN and NOBCKOF are 0. So you expect to read back 0x0000_0001. The restore in Step F puts MODER back to 0x0000_A000 before the next iteration.

Bit 11 — Reserved

k = 11

dc_mask[11] = 1 → continue (skip this bit entirely)

No write. No read. No compare.
The sequence just moves to k=12.

Bit 13 — CRCEN (reset = 1)

k = 13

Step A: wr_val = (1 << 13) = 0x0000_2000

Step B: WISHBONE write
ADDR_I = 0x00
DATA_I = 0x0000_2000 ← only CRCEN=1, everything else 0
WE_I = 1

Step C: WISHBONE read
DUT returns DATA_O = 0x0000_2000

Step D: exp_val = 0x0000_2000 (just CRCEN bit set)

Step E: 0x0000_2000 === 0x0000_2000 → PASS ✓

Step F: restore: write 0x0000_A000

The Walking-0 Pattern (Part 2 of the Same Sequence)

After all 32 walking-1 iterations for MODER, the sequence runs walking-0. First it writes all 1s to every RW bit:

Write all-RW-ones:
DATA_I = ~dc_mask = ~0xFFFF_0800 = 0x0000_F7FF

MODER now = 0x0000_F7FF
(every RW bit is 1: RECSMALL, PAD, HUGEN, CRCEN, DLYCRCEN,
FULLD, EXDFREN, NOBCKOF, LOOPBCK, IFG, PRO, IAM, BRO,
NOPRE, TXEN, RXEN all = 1)

Then for each RW bit position, it clears that one bit to 0:

Walking-0 for bit 0 (RXEN):

k = 0, val = 0

Step A: wr_val = ~(1 << 0) & ~dc_mask
= 0xFFFF_FFFE & 0x0000_F7FF
= 0x0000_F7FE
(every RW bit = 1 EXCEPT RXEN which is 0)

Step B: write 0x0000_F7FE to MODER

Step C: read back from DUT

Step D: exp_val = 0x0000_F7FE

Step E: compare → PASS ✓

Step F: restore to 0x0000_A000

Now Let Us See Every WISHBONE Transaction For Just IPGT

IPGT is simpler — only 7 bits, all RW, reset = 0x12 = 0001_0010.

dc_mask = 0xFFFF_FF80 (bits 31–7 are reserved or don't exist)

Walking-1 iterations: 7 total (bits 0–6)
Walking-0 iterations: 7 total (bits 0–6)
Total WISHBONE cycles: 7×(write+read+restore) + setup + 7×(write+read+restore)
= 7×3 + 1 + 7×3 = 43 WISHBONE cycles just for IPGT

Here is every transaction:

WALKING-1 FOR IPGT:

—— Bit 0 ——————————————————————————————————————
Write: ADDR=0x0C DATA=0x00000001 WE=1 SEL=0xF → ACK
Read: ADDR=0x0C WE=0 → DATA=0x00000001 ACK
compare 0x01 vs 0x01 → PASS
Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

—— Bit 1 ——————————————————————————————————————
Write: ADDR=0x0C DATA=0x00000002 WE=1 SEL=0xF → ACK
Read: ADDR=0x0C WE=0 → DATA=0x00000002 ACK
compare 0x02 vs 0x02 → PASS
Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

—— Bit 2 ——————————————————————————————————————
Write: ADDR=0x0C DATA=0x00000004 WE=1 SEL=0xF → ACK
Read: ADDR=0x0C WE=0 → DATA=0x00000004 ACK
compare 0x04 vs 0x04 → PASS
Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

—— Bit 3 ——————————————————————————————————————
Write: ADDR=0x0C DATA=0x00000008 WE=1 SEL=0xF → ACK
Read: ADDR=0x0C WE=0 → DATA=0x00000008 ACK
compare 0x08 vs 0x08 → PASS
Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

—— Bit 4 ——————————————————————————————————————
Write: ADDR=0x0C DATA=0x00000010 WE=1 SEL=0xF → ACK
Read: ADDR=0x0C WE=0 → DATA=0x00000010 ACK
compare 0x10 vs 0x10 → PASS
Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

—— Bit 5 ——————————————————————————————————————
Write: ADDR=0x0C DATA=0x00000020 WE=1 SEL=0xF → ACK
Read: ADDR=0x0C WE=0 → DATA=0x00000020 ACK
compare 0x20 vs 0x20 → PASS
Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

—— Bit 6 ——————————————————————————————————————
Write: ADDR=0x0C DATA=0x00000040 WE=1 SEL=0xF → ACK
Read: ADDR=0x0C WE=0 → DATA=0x00000040 ACK
compare 0x40 vs 0x40 → PASS
Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

Bits 7–31: all in dc_mask → skipped entirely

WALKING-0 SETUP FOR IPGT:

Write: ADDR=0x0C DATA=0x0000007F WE=1 SEL=0xF → ACK
    (all 7 RW bits = 1: 0111_1111)

—— Bit 0 (RXEN walking-0) ——————————————————————————
Write: ADDR=0x0C DATA=0x0000007E WE=1 SEL=0xF → ACK
    (0111_1111 with bit 0 cleared = 0111_1110)
Read: ADDR=0x0C WE=0 → DATA=0x0000007E ACK
        compare 0x7E vs 0x7E → PASS
Write: ADDR=0x0C DATA=0x00000012 WE=1 → ACK (restore)

... (repeat for bits 1–6)

What Bugs This Test Catches

Each failure points to a very specific RTL problem:

Failure type	What it means in RTL
Read back all zeros always	Register not connected to DATA_O path Wrong address decode in eth_registers.v
Read back all ones always	DATA_O stuck high Missing output enable
Bit N always reads 0	That specific flip-flop not written Bit N of DATA_I not connected to FF input
Bit N always reads 1	FF reset value wrong, or bit stuck
Walking-1 PASS but walking-0 FAIL on same bit	Register has bit coupling — writing one bit affects another (logic bug in DUT)

Reserved bits change when RW bits written	Reserved bit positions not properly masked in eth_registers.v
Value does not restore after Step F	Restore write not working (same as write failure but only on the reset value pattern)
ERR_O asserts on valid register address	SEL_I checking too strict, or address decode bug in eth_wishbone.v

Total Bus Transactions Across All 21 Registers

Per register:
- Walking-1: (number of RW bits) × 3 transactions (write + read + restore)
- Walking-0: 1 setup write + (number of RW bits) × 3 transactions

For your 21 registers (approximate RW bit counts):

MODER	15 RW bits → 15×3 + 1 + 15×3 = 91
INT_SOURCE	7 RW bits → 7×3 + 1 + 7×3 = 43
INT_MASK	7 RW bits → 43
IPGT	7 RW bits → 43
IPGR1	7 RW bits → 43
IPGR2	7 RW bits → 43
PACKETLEN	32 RW bits → 32×3 + 1 + 32×3 = 193
COLLCONF	14 RW bits → 85
TX_BD_NUM	8 RW bits → 49
CTRLMODER	3 RW bits → 19
MIIMODER	9 RW bits → 55
MIICOMMAND	3 RW bits → 19
MIIADDRESS	10 RW bits → 61
MIITX_DATA	16 RW bits → 97
MAC_ADDR0	32 RW bits → 193
MAC_ADDR1	16 RW bits → 97
HASH0	32 RW bits → 193
HASH1	32 RW bits → 193
TXCTRL	17 RW bits → 103

Total ≈ 1,663 WISHBONE transactions

At ~3 clock cycles each with no wait states ≈ 5,000 clock cycles
At 100MHz system clock = 50 microseconds of simulation time

This is why `uvm_reg_bit_bash_seq` is always run early in regression — it is fast, completely automatic, and catches a huge class of RTL connectivity bugs before you even attempt functional tests.

3- tc_bd_walking_one

Exercises all 1024 bytes of the internal BD RAM using uvm_mem_walk_seq. Verifies that the RAM correctly stores and returns data at every address without corruption. BD RAM is only accessible when the MAC is not in reset.

COMPONENTS USED

`wb_slave_agent` `eth_bd_mem (uvm_mem)` `uvm_mem_walk_seq`

TEST FLOW

1	`Test`	Ensure MAC is not in reset (RST bit in MODER cleared)
2	`uvm_mem_walk_seq`	Writes walking-1 pattern to every 32-bit word from offset 0x400 to 0x7FC
3	`wb_slave_driver`	Each write: ADDR_I = 0x400+offset, DATA_I = pattern, WE_I=1, SEL_I=0xF
4	`uvm mem walk seq`	Reads back every location and compares
5	`Test`	Assert RST_I, then read BD RAM — verify values preserved

PASS CRITERIA

All 256 32-bit words (0x400–0x7FC) store and return correct values
No ACK timeout for any BD address within 0x400–0x7FF range
Addresses outside 0x400–0x7FF do not alias into BD RAM
RAM values preserved across soft reset (RST_I) as per spec
READY bit (RD/E) prevents host access once set — verify ERR_O or no effect when written

4- tc_tx_bd_num_split

Writes 0x32 (50) to TX_BD_NUM register and verifies that exactly 50 TX BDs are accessible at 0x400–0x58C and 78 RX BDs are accessible at 0x590–0x7FC. Tests the boundary address for each region.

COMPONENTS USED

`wb_slave_agent` `eth_reg_block (RAL)` `eth_bd_mem (uvm_mem)` `eth_tx_bd_num_seq`

TEST FLOW

1	`eth_tx_bd_num_seq`	Write TX_BD_NUM = 0x32 via RAL frontdoor
2	`eth_tx_bd_num_seq`	Write all 50 TX BDs: address range 0x400 to 0x58C (8 bytes each × 50 = 0x190)
3	`eth_tx_bd_num_seq`	Write all 78 RX BDs: address range 0x590 to 0x7FC
4	`eth_tx_bd_num_seq`	Read back all 128 BDs and verify no overlap or corruption
5	`Test`	Write TX_BD_NUM = 0x80 (all TX): verify RXEN auto-disabled in MODER
6	`Test`	Write TX_BD_NUM = 0x00 (all RX): verify TXEN auto-disabled in MODER

PASS CRITERIA

TX BD at 0x400 (first) and 0x58C (last) correctly stored and read
RX BD at 0x590 (first) and 0x7FC (last) correctly stored and read
RXEN auto-disables when TX_BD_NUM = 0x80
TXEN auto-disables when TX_BD_NUM = 0x00
Values > 0x80 are ignored (register retains previous value)

5- tc_miim_write_phy_reg

Programs a PHY configuration register by setting up MIIADDRESS, MIITX_DATA, then asserting WCTRLDATA in MIICOMMAND. Verifies the full 64-bit MIIM frame appears on the MDC/MDIO interface with correct preamble, address, and data fields.

COMPONENTS USED

`wb_slave_agentmdio_agent` `eth_reg_block (RAL)` `eth_miim_write_seq` `mdio_monitor` `eth_scoreboard`

TEST FLOW

1	`eth_miim_write_seq`	RAL write: MIIMODER.CLKDIV = 0x28 (MDC = CLK/40)
2	`eth_miim_write_seq`	RAL write: MIIADDRESS.FIAD = 0x01 (PHY addr), RGAD = 0x00 (control reg)
3	`eth_miim_write_seq`	RAL write: MIITX_DATA.CTRLDATA = 0x2100
4	`eth_miim_write_seq`	RAL write: MIICOMMAND.WCTRLDATA = 1
5	`wb slave driver`	Issues all four WISHBONE write cycles with ACK_O handshake
6	`mdio_monitor`	Captures MDC edges and samples MDIO bit stream
7	`mdio_monitor`	Decodes: 32-bit preamble(1s), ST=01, OP=01(write), FIAD, RGAD, TA=10, DATA
8	`eth_miim_write_seq`	poll MIISTATUS.BUSY via RAL read until cleared
9	`mdio_agent`	Simulates PHY acknowledging write (drives MDIO during TA)
10	`eth_scoreboard`	Verifies decoded frame fields match programmed values

PASS CRITERIA

MDIO preamble = 32 ones (unless MIINOPRE set)
ST field = 2'b01
OP field = 2'b01 for write
FIAD[4:0] matches MIIADDRESS.FIAD written
RGAD[4:0] matches MIIADDRESS.RGAD written
TA = 2'b10 for write
DATA[15:0] matches MIITX_DATA.CTRLDATA
MIISTATUS.BUSY deasserts at end of operation

6- tc_miim_clkdiv_all

Iterates through all valid CLKDIV values (1–255) and for each value triggers a MIIM read operation, measures the MDC frequency, and verifies it equals CLK/(2×CLKDIV).

COMPONENTS USED

`wb_slave_agentmdio_agent` `eth_reg_block (RAL)` `eth_miim_clkdiv_seq`

TEST FLOW

1	`eth_miim_clkdiv_seq`	For CLKDIV = 1 to 255: RAL write to MIIMODER.CLKDIV
2	`eth_miim_clkdiv_seq`	Trigger MIICOMMAND.RSTAT = 1 to start a read operation
3	`mdio_monitor`	Measures MDC period: records rising edge timestamps, computes frequency
4	`mdio_monitor`	Verifies MDC_freq = SYS_CLK_freq / (2 × CLKDIV) ± 1 cycle tolerance
5	`eth_miim_clkdiv_seq`	Wait for BUSY = 0 before next iteration

PASS CRITERIA

MDC frequency matches formula for all 255 values
No MDC glitches (spurious edges) between operations
BUSY correctly asserted for full duration of each read
CLKDIV=0 treated as 1 or ignored (per implementation)

We need read operation as MDC does not run continuously. It only toggles when the MIIM module is actively performing an operation.

MDC is a divided version of CLK. The counter counts CLKDIV system clock cycles, then toggles MDC. One full MDC period = 2 × CLKDIV system clock cycles.

7- tc_miim_scan_linkfail

Asserts SCANSTAT in MIICOMMAND to start continuous PHY status polling. The mdio_agent (PHY model) first returns link-OK, then drives LINKFAIL=1 in the status register. Verifies MIISTATUS.LINKFAIL reflects the PHY state and NVALID clears after first scan completes.

COMPONENTS USED

wb_slave_agent mdio_agent	eth_reg_block (RAL)	eth_miim_scan_seq

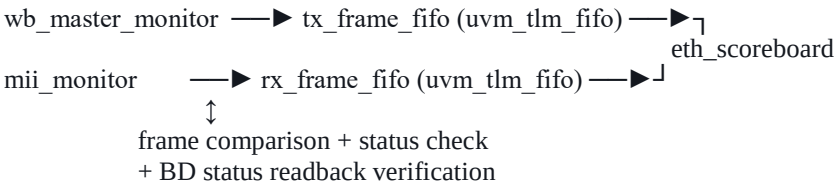
TEST FLOW

1	eth_miim_scan_seq	Write MIICOMMAND.SCANSTAT = 1; MIIADDRESS.FIAD = PHY_ADDR
2	mdio_agent	First scan response: bit[2]=1 (link OK) in PHY status register
3	eth_miim_scan_seq	RAL read MIISTATUS: verify NVALID=1 (invalid until first scan done)
4	mdio_monitor	Detects end of first scan frame; NVALID should clear
5	eth_miim_scan_seq	RAL read MIISTATUS: verify NVALID=0, LINKFAIL=0
6	mdio_agent	Second scan response: bit[2]=0 (link fail) in PHY status register
7	eth_miim_scan_seq	RAL read MIISTATUS: verify LINKFAIL=1
8	eth_miim_scan_seq	Write MIICOMMAND = 0x00 to stop scan; verify BUSY deasserts

PASS CRITERIA

NVALID=1 until first complete scan frame received
NVALID=0 after first scan completes
LINKFAIL tracks PHY bit[2] of status register
BUSY stays asserted during entire scan operation
BUSY deasserts only after scan explicitly stopped

5. Scoreboard Architecture



The scoreboard:

- Reconstructs full frames from WISHBONE master writes (TX path) and MII nibble captures (RX path)
- Verifies CRC correctness, frame length, padding
- Compares BD status fields after each transfer (RTRY, RL, LC, DF, CS for TX; CRC, SF, TL, OR, IS, DN for RX)
- Checks interrupt generation matches enabled masks in INT_MASK